

ELF-WONN 核心算法简报

聚焦当前 ELF-WONN 实现中振荡器 backbone 的状态、耦合动力学与读出过程。

技术摘要

ELF-WONN 用 **Winfree 振荡器动力学完整替换 ELF 的 Transformer denoiser backbone**，但不改变 ELF 的连续 latent、Flow Matching 或采样接口。每次 denoiser 前向都会把当前 token hidden state 确定性地初始化为相位 θ 与内在频率 ω ；相位在 attentive Winfree coupling 下快速演化，频率只在相邻层之间慢速更新，最终仅从相位的周期特征 $[\sin \theta, \cos \theta]$ 读出结果。

正式 ELF-WONN-B 为：每个 token **384 个振荡器**、**12 个 coupling heads**、**6 层 × 每层 2 个 inner steps**，因此每次前向进行 **12 次 coupling evaluation**；层间共有 **5 次 frequency transition**。

Source: WONN 模型主体

File: src/modules/wonn_model.py

1. 状态表示与确定性初始化

对 batch 中第 i 个 token，WONN 的内部状态为

$$\begin{aligned}\theta_i, \omega_i &\in \mathbb{R}^K, \quad K = 384 \\ P(\theta_i) &= [\sin \theta_i, \cos \theta_i] \in \mathbb{R}^{(2K)}\end{aligned}$$

ELF 的输入、时间 token、self-conditioning/CFG token 和 mode token 先按原接口组成 hidden sequence H 。归一化后，用两个投影初始化：

$$\begin{aligned}(a_{i,k}, b_{i,k}) &= W_\theta H_i \\ \theta_{i,k} &= \text{atan2}(b_{i,k}, a_{i,k}) \\ \omega_{i,k} &= W_\omega H_i\end{aligned}$$

每个 (a, b) 对会先单位化，再用 atan2 得到相位。初始化不引入随机数；下一次 flow/sampling step 会从新的当前 latent 重新计算 (θ, ω) ，不会跨 denoiser 调用携带隐藏状态。

整体路径： ELF hidden $H \rightarrow (\theta^0, \omega^0) \rightarrow L$ 个 WONN layers $\rightarrow$ final phase features $\rightarrow$ ELF-compatible output。

Source: WONN 模型主体

File: src/modules/wonn_model.py

2. Attentive Winfree 单步动力学

384 个振荡器被分成 12 组，每个 head 管理 32 个通道。对第 1 层、第 s 个 inner step，先从周期特征计算带 RoPE 的 query/key：

```
Ai,j,h = softmaxj( qi,h · kj,h / sqrt(dqk) )
```

注意力 A 只表示 token 间的耦合权重。每个 head 再从相位产生接收端敏感度 S 和发送端影响 I：

```
Si,h = tanh(WhS [sin θi,h, cos θi,h] + bhS)  
Ij,h = tanh(WhI [sin θj,h, cos θj,h] + bhI)  
mi,h = Σj Ai,j,h Ij,h  
vi,h = ωi,h + Si,h ⊙ mi,h
```

最后用有界、可学习步长推进相位：

```
γl = γmax · sigmoid(rl),    γl 初值 0.1, γmax = 0.25  
θi,h ← wrap(θi,h + γl vi,h),    wrap 范围为 [-π, π]
```

padding mask 直接作用在 A 上。这里没有 Transformer 的 value projection residual，也没有 FFN；attention 只参数化 Winfree coupling，真正改变状态的是上面的相位方程。

Source: WONN 动力学层

File: src/modules/wonn_layers.py

3. 快相位、慢频率与 phase-only 读出

一个 WONN layer 内，ω 在 2 个 inner steps 中保持不变；完成相位迭代后，仅在还存在下一层时执行 gated residual frequency transition：

```
δωl = tanh(Wωl RMSNorm([sin θl, cos θl, ωl]))  
ω(l+1) = ωl + tanh(gl) · δωl  
θ(l+1,0) = θ(l,T)
```

g_l 从 0 初始化，因此模型训练初期近似固定内在频率，之后再学习跨层的慢速变化。相位在层边界直接传递，不经过额外坐标变换。最后一层之后不再更新 ω，因为输出只读取最终相位：

```
Hout = [sin θ(L,T), cos θ(L,T)]  
ycontinuous = FinalLayer(Hout)  
ydecode = Unembed(GELU(Hout Wproj + bproj))    # 仅 decode mode
```

控制 token 在动力学中参与耦合，但读出前会被移除；返回的 continuous output 与 decoder logits 仍保持 ELF 原有 shape 和 dtype 契约。

Source: WONN 动力学层

File: src/modules/wonn_layers.py

层间频率更新日程

Source: 逐层更新日程

Table: layer_schedule

WONN 层	层后频率更新次数	层内相位更新次数
L1	1	2
L2	1	2
L3	1	2
L4	1	2
L5	1	2
L6	0	2

4. 精简伪代码

```
H, mask, n_ctrl = PrepareELFSequence(x, t, self_cond, mode)
θ, ω = DeterministicInit(H)

for l = 1 ... L:
    for s = 1 ... T:
        A = MaskedAttention(P(θ), RoPE)
        message = A @ Influence(P(θ))
        velocity = ω + Sensitivity(P(θ)) * message
        θ = wrap(θ + step_l * velocity)

    if l < L:
        ω = ω + gated_frequency_delta(P(θ), ω)

H_out = P(θ)[: , n_ctrl:]
return ContinuousHead(H_out), OptionalDecodeHead(H_out)
```

Source: WONN 模型主体

File: src/modules/wonn_model.py

5. 汇报时必须明确的边界

- **定位：** WONN 是 denoiser backbone，不是新的 latent space； θ/ω 只存在于一次前向内部。
- **机制：** attention 只产生耦合权重，没有标准 A·V residual 或 FFN。
- **核心归纳偏置：** θ 每个 inner step 更新， ω 只在层间更新，最终仅从相位读出。
- **比较边界：** 12 次 coupling evaluation 只是结构次数对齐，不能直接推出 FLOPs、延迟或任务质量更优。

Source: Phase 3 WONN-ELF 结构说明

File: docs/PHASE3_WONN_ELF.md